

My AI Workflow & Setup

Interview prep — questions & answers

QUESTION 1

What does my AI workflow / usual setup look like? What tools do I use?

Full answer (2-min)

My setup is agent-first and multi-engine. I run several **agentic coding CLIs** side by side, pick the right one per task, and drive them all with a disciplined workflow layer.

The engines. My daily driver is **Claude Code** on the latest Claude models. Alongside it I run **OpenAI Codex** and **opencode** — and the piece I lean on most for orchestration is **Hermes**, a self-hosted multi-provider agent platform. Hermes is the interesting one: not just a chat CLI, a full agent runtime. It does **provider routing and automatic fallback** — if my primary model fails it retries on another — plus a **Mixture-of-Agents** mode where several models collaborate on one answer. It has its own **gateway with Slack and WhatsApp bridges, cron for scheduled agents, a kanban board** for task state, **LSP integration**, and a **memory graph** for persistent context. So it is less ‘a tool’ and more my personal agent infrastructure. For fully local / private runs I drop to **LM Studio**.

The workflow layer. I do not treat any of this as autocomplete. I run a methodology called **Superpowers** — a set of process skills that enforce real engineering discipline: brainstorm the design before coding, write a plan before touching files, test-driven development, and systematic debugging instead of guess-and-check. It also handles parallel-agent dispatch and code-review checkpoints. The result is that agents follow a repeatable process, not vibes.

My own skills — built around the latest tech. On top of the shared frameworks I maintain my own skill marketplace, focused on current, fast-moving tech — new model capabilities, the latest frameworks, MCP tooling. That keeps my agents working against how things are *today*, not stale training data. When something new drops, I encode it as a skill rather than re-explaining it every session.

Quality + context. Git hooks block dangerous commands, validate commits, and scan for prompt-injection in anything an agent reads; dedicated review and simplifier agents check the diff before merge. And models connect to real context via **MCP servers** — live library docs (Context7), semantic code navigation (Serena), design tools (Figma / Stitch), GitHub / Linear / Notion.

Net-net: I treat AI as a **fleet of specialized agents with a real process** — Hermes orchestrates and routes, Claude Code does the heavy engineering, Superpowers keeps it disciplined, and my own

skills keep the whole thing current with the latest tech.

30-second version

I run multiple agentic CLIs — **Claude Code** for heavy engineering, **Codex** and **opencode** alongside — orchestrated through **Hermes**, a self-hosted multi-provider agent with automatic fallback, Mixture-of-Agents, scheduled cron agents, and Slack / WhatsApp gateways. I drive them with **Superpowers**, a workflow methodology that enforces brainstorm → plan → TDD → systematic debugging. And I maintain my own **latest-tech skill library** so the agents work against current frameworks and model capabilities. Plus git guardrails, review agents, and MCP servers for live docs and semantic code search.

One-liner

A self-hosted agent stack — **Hermes** for multi-provider orchestration, **Claude Code / Codex / opencode** as engines, **Superpowers** for engineering discipline, and my own **latest-tech skill library** keeping it all current.

Defending the ‘own skills, latest tech’ angle

“Instead of re-explaining a new framework or MCP pattern to the AI every time, I write it once as a reusable skill. So when a new model or tool ships, my agents pick it up immediately — my setup stays current instead of decaying to whatever the model was trained on months ago.”

QUESTION 2

How do you adapt to new and upcoming technologies and methodologies — like skills — in AI?

Full answer

My honest take is that in AI the half-life of knowledge is short — what is true about a model or framework this quarter may not hold next quarter. So I do not try to *memorize* the ecosystem; I build a **system that absorbs new tech quickly** and keeps my tooling current by default. Concretely, three habits:

- **I encode knowledge instead of re-learning it.** When a new framework, model capability, or pattern drops, I turn it into a reusable **skill** my agents can invoke. Same instinct as writing a good internal doc or a library wrapper: learn it once, encode it, and it compounds. My setup stays

pinned to how things actually work today rather than decaying to stale training data.

- **I keep the AI grounded in live sources, not memory.** I wire models to **MCP servers** — Context7 for live library docs, Serena for semantic code navigation. When I work with a fast-moving library, the agent reads the current docs instead of guessing. Adapting to a new version becomes automatic instead of a research project.
- **I stay provider- and tool-agnostic on purpose.** I run multiple engines — Claude Code, Codex, opencode, orchestrated through Hermes with automatic fallback. A deliberate hedge: no single model or vendor stays ahead forever, so I keep the architecture swappable and can adopt a better model or methodology without rebuilding.

The underlying methodology matters more than any specific tool. I lean on **Superpowers-style discipline** — brainstorm and plan before coding, TDD, systematic debugging. Those fundamentals do not change when the tools do, so they give me a stable base to plug new tech into.

So my adaptation strategy is basically: **stable process, swappable tools, and encode-what-you-learn**. New tech is not a disruption to catch up on — it is just a new skill or a new MCP server I slot into a system built to change.

Short version (30s)

AI moves too fast to memorize, so I built a system that adapts instead. When new tech drops, I encode it as a reusable **skill** rather than re-learning it each time. I keep the AI grounded in **live docs via MCP** instead of stale training data. And I stay **provider-agnostic** — multiple engines with fallback — so I can adopt a better model or new methodology without rebuilding. The constant is the **process** — plan, TDD, systematic debugging — which stays stable while the tools swap underneath.

The concrete example

*“Take **MCP** — the model-context protocol. When it emerged, instead of manually pasting docs into every session, I stood up MCP servers so my agents pull live library docs and navigate code semantically. Same with **agent-skills**: the first time I explained a new framework to the AI, I saved it as a skill — so the second time, and every time after, it just worked. That is the pattern: the first encounter with new tech becomes a permanent capability, not a repeated cost.”*

Why this answer lands

- Reframes ‘how do you keep up’ from anxious catching-up to engineered adaptability — a maturity signal.
- **‘Stable process, swappable tools’** is the memorable soundbite — it shows you separate fundamentals from fashion.

- Ties directly to a real setup, so follow-ups are easy to defend.